

PLANO DE ARQUITETURA & EFICIÊNCIA DE TOKENS

Automação do Repomix com Git Hooks & Direcionamento de Harness

Estratégia para corte de até 85% do custo de contexto em sessões com agentes de IA

1. Objetivo Executivo

Este documento estabelece o plano formal para automatizar o ciclo de empacotamento de contexto via **Repomix**, integrando **Git Hooks** (pré e pós-commit) e regras comportamentais de **Harness** (Antigravity, Cursor, Claude Code, Aider).

A meta é eliminar a exploração cega do repositório por agentes autônomos, consolidando a árvore de símbolos em um único payload estruturado em XML com numeração de linhas e aproveitamento máximo de **Prompt Caching**.

2. Arquitetura do Fluxo Operacional

FLUXO A · Pré-Ativação do Agente (Context Hook / Governança)

- Prompt do Operador:** O usuário dispara uma solicitação de refatoração ou criação de feature.
- Intercepção de Governança:** A regra em `AGENTS.md` e a Skill `lean-ctx` impedem o agente de rodar varreduras cegas (`list_dir`, múltiplos `view_file`).
- Leitura Única do Snapshot:** O agente consome `repomix-output.xml` em 1 única chamada estruturada.
- Ativação de Prompt Caching:** As LLMs (Anthropic, OpenAI, Gemini) aplicam desconto de até 90% no cache de prefixo nos turnos seguintes.

FLUXO B · Pós-Modificação / Ciclo Git (Post-Hook Automático)

- Entrega do Código:** O operador ou o subagente finaliza as alterações e executa o `git commit`.
- Disparo do Hook:** O hook `.git/hooks/post-commit` é acionado de forma assíncrona.
- Regeneração Incremental:** O script executa `npx repomix` em menos de 1 segundo, atualizando o snapshot.
- Estado em Disco Atualizado:** O repositório está pronto e sincronizado para a próxima sessão de IA.

3. Detalhamento das 3 Fases de Implementação

Fase 1: Configuração Fina do Repomix (`repomix.config.json`)

Criação de um arquivo de configuração declarativo na raiz para isolar apenas o código essencial:

- Inclusões Estratégicas:** `src/**`, `scripts/**`, `.claude/skills/**`, `docs/**`.
- Exclusões Rígidas:** `package-lock.json`, `node_modules`, pastas de build (`dist/`, `output/`, `.git/`), assets pesados e binários.
- Formato de Saída:** xml com numeração de linhas (`output.parsableStyle = true`) para eliminar alucinações de arquivos.

Fase 2: Automação via Git Hooks (Pós-Commit)

- **Hook:** scripts/hooks/post-commit copiado para .git/hooks/post-commit.
- **Script Executor:** scripts/gerar-contexto-repomix.py com execução determinística e tratamento de erro silencioso em background.
- **Idempotência:** A reexecução do hook não corrompe o estado e roda em microssegundos.

Fase 3: Direcionamento do Harness & Governança

- **Regra em AGENTS.md:** Proíbe explicitamente chamadas em loop de ferramentas de busca quando o snapshot estiver disponível.
- **Skill repomix-navigator:** Injeta a diretiva de priorizar a consulta ao XML empacotado para mapeamento de dependências e contratos.

4. Matriz de Entregáveis

Componente	Tipo	Localização	Responsabilidade
repomix.config.json	Config	/repomix.config.json	Filtros, exclusões e estilo XML.
post-commit	Git Hook	scripts/hooks/post-commit	Regenera o snapshot após cada commit.
gerar-contexto.py	Script	scripts/gerar-contexto.py	Execução local do Repomix em background.
Regra no AGENTS.md	Governança	/AGENTS.md	Força o agente a usar o snapshot antes de explorar.